SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA1408 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool *(BL3)*

Assessment Tool Weightage: 20%

Duration :60 Mins
On class
QUESTIONS: **Total Marks:50**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I Ranjitha. R (192421285) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

ERROR ANALYSIS TASK

Q1. The following C program is intended to perform string processing, function calls, array manipulation, conditional execution, pointer operations, switch-case selection, and loop execution. However, the program contains several errors introduced during different phases of compilation, including lexical, syntax, semantic, logical, and runtime errors.

Given Program

```
#include <stdio.h>
#include <string.h>

int findLength(char str[])
{
   return strlen(str);
}

int main()
{
   char name[20] = "Compiler"

   int count;

   count = findLength();

   if(count > 5
   {
      printf("Length is greater than 5\n");
   }

   int marks[4] = {85, 90, 75, 80};

   printf("%d\n", marks[4]);

   int choice = 2;

   switch(choice)
   {
      case 1:
         printf("Option 1\n");
         break;

      case 2
         printf("Option 2\n");
         break;

      default:
         printf("Invalid Choice\n");
```

```
    }

    int *ptr;
    printf("%d\n", *ptr);

    int total = 100;
    int students = 0;

    float average = total / students;

    int i = 1;

    while(i <= 5)
    {
        printf("%d\n", i);
    }

    return 0;
}
```

Tasks

(a) Identify all errors in the program. Mention the line number, error type, and reason for each error. (5 Marks)

(b) Classify the identified errors as Lexical, Syntax, Semantic, Logical, or Runtime errors in a suitable table. (5 Marks)

(c) Explain at which phase of compilation each category of error is detected. Also mention which errors are detected only during execution. (4 Marks)

(d) Rewrite the program after correcting all the errors so that it executes successfully. (4 Marks)

(e) Suggest any three compiler features or warnings that can help detect such programming errors before execution. (2 Marks)

SOLUTION:

(a) Identify all errors

Line	Error	Type	Reason
11	Missing ; After "Compiler"	Syntax	Statement not terminated
15	findLength();	Semantic	Function requires one argument
17	Missing) in if(count > 5	Syntax	Parenthesis not closed
26	marks[4]	Runtime	Array index out of bounds (valid: 0–3)
35	Missing : after case 2	Syntax	Invalid switch syntax

45	*ptr without initialization	Runtime	Dereferencing garbage pointer
50	total / students where students=0	Runtime	Division by zero
55	i never incremented	Logical	Infinite loop

(b) Classification

| Error	**Category**
Missing ;	Syntax
Missing)	Syntax
Missing : after case	Syntax
Missing function argument	Semantic
Array out of bounds	Runtime
Uninitialized pointer	Runtime
Division by zero	Runtime
nfinite loop	Logical

(c) Error detection phase

Error Category |	**Detected During**
Lexical	Lexical Analysis
Syntax	Syntax Analysis (Parser)
Semantic	Semantic Analysis
Logical	Not detected by compiler (Program execution/testing)
Runtime	During execution by OS/runtime environment

Runtime-only errors:

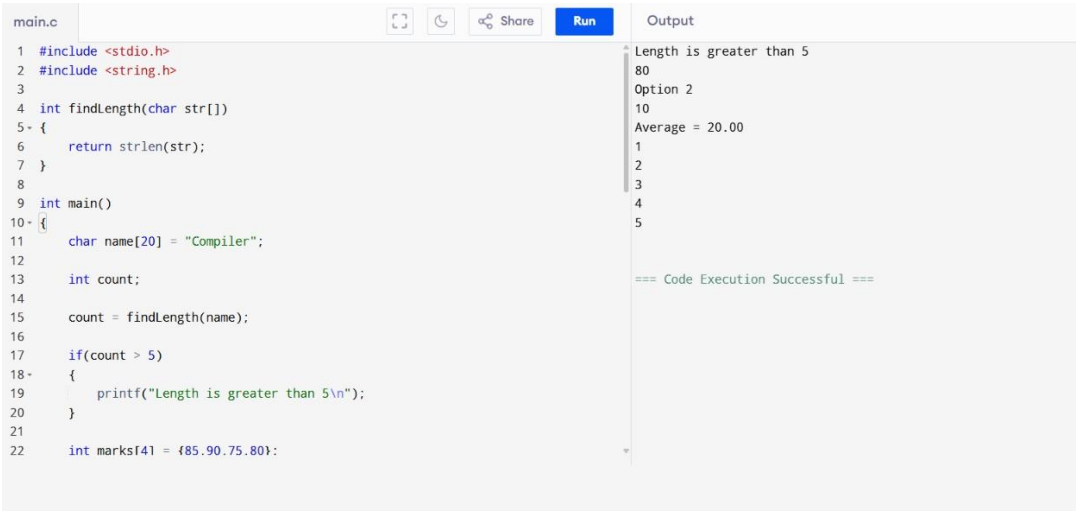
- Array out of bounds
- Division by zero
- Uninitialized pointer

(d) Correct Program

```
#include <stdio.h>
#include <string.h>

int findLength(char str[])
{
   return strlen(str);
}

int main()
```

```
{
   char name[20] = "Compiler";

   int count;

   count = findLength(name);

   if(count > 5)
   {
      printf("Length is greater than 5\n");
   }

   int marks[4] = {85,90,75,80};

   printf("%d\n", marks[3]);

   int choice = 2;

   switch(choice)
   {
      case 1:
         printf("Option 1\n");
         break;

      case 2:
         printf("Option 2\n");
         break;

      default:
         printf("Invalid Choice\n");
   }

   int value = 10;
   int *ptr = &value;
   printf("%d\n", *ptr);

   int total = 100;
   int students = 5;

   float average = (float)total / students;
   printf("%.2f\n", average);

   int i = 1;
```

```
    while(i <= 5)
    {
        printf("%d\n", i);
        i++;
    }

    return 0;
}
```



(e) Three compiler features

1. Enable -Wall warnings.
2. Enable -Wextra warnings.
3. Use *Static Code Analysis* (Clang Static Analyzer).

Q2. The following C program is intended to perform function calls, array processing, conditional execution, switch-case selection, and loop operations. However, several syntax errors have been intentionally introduced by violating the grammatical rules of the C language. These errors include missing delimiters, unmatched parentheses, incorrect control structures, malformed function definitions, and incomplete statements. Such errors are normally detected during the syntax analysis (parsing) phase of a compiler.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>

int maximum(int x, int y
{
    if(x > y)
        return x;
```

```
    else
        return y;
}

int main()
{
    int marks[5] = {70, 80, 90, 85, 95};

    int max = maximum(marks[0], marks[1]);

    switch(max)
    {
        case 80:
            printf("Maximum is 80\n")
            break;

        case 90
            printf("Maximum is 90\n");
            break;

        default:
            printf("Other Value\n");
    }

    do
    {
        printf("%d\n", max);
        max--;
    while(max > 75);

    if(max == 75)
        printf("Limit Reached\n"
    }

    return 0;
```

Tasks

(a) Identify all syntax errors in the program. Mention the line number and reason for each error. (4 Marks)

(b) Classify the identified syntax errors under the appropriate categories and present them in a table. (3 Marks)

(c) Correct the syntax errors and rewrite the complete program. (4 Marks)

(d) Explain how the parser detects syntax errors and how error recovery enables further parsing. (2 Marks)

(e) Suggest any two improvements that can help a compiler generate better syntax error messages. (2 Marks)

SOLUTION:

(a) Syntax errors

Line	Error	Reason
3	Missing) in function header	Invalid function declaration
16	Missing ; after printf()	Statement not terminated
19	Missing : after case 90	Invalid case syntax
31	Missing) in printf()	Parenthesis not closed
32	Extra }	Block mismatch
34	Missing } for main()	Function not closed properly

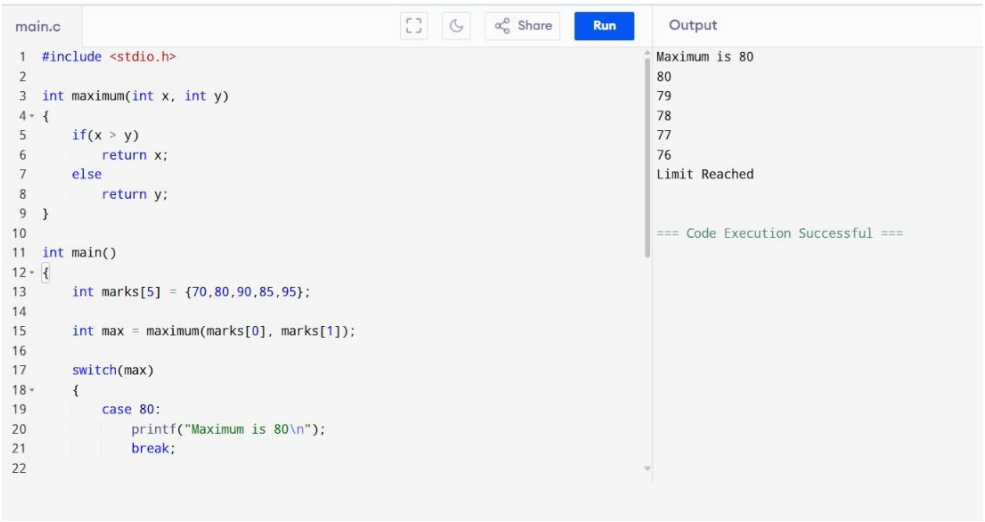
(b) Classification

Error	Category
Missing)	Parentheses error
|Missing ;	Statement termination
|Missing :	Switch-case syntax
Missing }	Block delimiter
Unmatched braces	Block structure

(c) Correct Program

```
#include <stdio.h>

int maximum(int x, int y)
{
   if(x > y)
      return x;
   else
```

```
    return y;
}

int main()
{
   int marks[5] = {70,80,90,85,95};

   int max = maximum(marks[0], marks[1]);

   switch(max)
   {
      case 80:
         printf("Maximum is 80\n");
         break;

      case 90:
         printf("Maximum is 90\n");
         break;

      default:
         printf("Other Value\n");
   }

   do
   {
      printf("%d\n", max);
      max--;
   }while(max > 75);

   if(max == 75)
      printf("Limit Reached\n");

   return 0;
}
```



(d) Parser and error recovery

- The parser checks whether the program follows the grammar of the C language.
- When a syntax error is found, error recovery skips or inserts tokens so parsing can continue and report multiple errors instead of stopping after the first one.

(e) Two improvements

1. Provide clear error messages with exact line numbers.
2. Suggest possible corrections (missing ;,), } etc.).

Q3. The following C program is intended to perform factorial computation, function calls, array processing, pointer manipulation, conditional execution, switch-case selection, and loop operations. However, the program contains several errors introduced during different phases of compilation and execution, including lexical, syntax, semantic, intermediate code generation, code optimization, and runtime errors.

Carefully examine the program and answer the questions that follow.

Given Program

```
#include <stdio.h>

int factorial(int n)
{
   if(n <= 1)
      return 1;

   return n * factorial(n - 1);
}
```

```
int main()
{
   int @num = 5;

   int value = 10

   int result;

   result = factorial();

   if(result >= 100)
   {
      printf("Large Number\n");
   }

   switch(result)
   {
      case 24:
         printf("Factorial of 4\n");
         break;

      case 120
         printf("Factorial of 5\n");
         break;

      default:
         printf("Other Value\n");
   }

   int numbers[5] = {2,4,6,8,10};
   printf("%d\n", numbers[5]);

   int *ptr;
   printf("%d\n", *ptr);

   int a = 50;
   int b = 0;
   int c = a / b;

   int sum = result + 0 + value;

   int i = 1;
   while(i <= 5)
   {
```

```
        printf("%d\n", i);
    }

    return 0;
}
```

Tasks

(a) Identify all the errors in the program. Mention the line number, error type, and reason for each error. (4 Marks)

(b) Classify the identified errors according to the appropriate compiler phase (Lexical, Syntax, Semantic, Intermediate Code Generation, Code Optimization, and Runtime) using a suitable table. (4 Marks)

(c) Explain how the compiler processes the program by identifying the errors detected during each compilation phase and those detected only at runtime. (3 Marks)

(d) Rewrite the complete corrected program so that it compiles and executes without errors. (2 Marks)

(e) Suggest any two improvements in compiler design or optimization techniques that could detect or prevent such errors before program execution. (2 Marks)

(a) Identify errors

Line	Error	Type	Reason
13	@num	Lexical	@ is an invalid identifier character
15	Missing ; after value=10	Syntax	Statement incomplete
19	factorial()	Semantic	Missing required argument
30	Missing : after case 120	Syntax	Invalid switch syntax
39	numbers[5]	Runtime	Array index out of bounds
42	*ptr	Runtime	Pointer not initialized
46	a/b where b=0	Runtime	Division by zero
52	Missing i++	Logical	Infinite loop

48	result + 0 + value	Code Optimization	+0 is redundant and can be optimized away

(b) Classification

Compiler Phase	Error
Lexical	Invalid identifier @num
Syntax	Missing ;, missing :
Semantic	Missing function argument
Intermediate Code Generation	Invalid program due to earlier syntax/semantic errors prevents code generation
Code Optimization	Redundant expression +0
Runtime	Array overflow, division by zero, uninitialized pointer
|Logical	Infinite loop

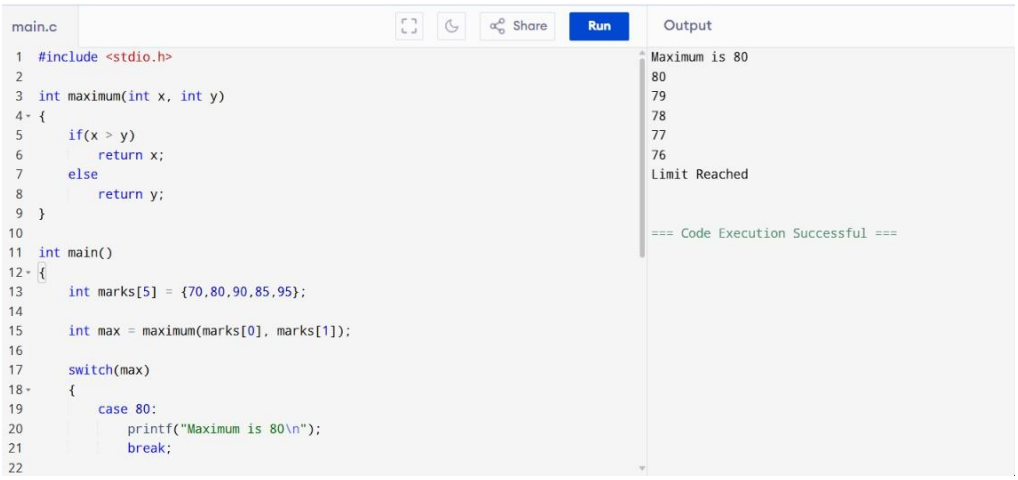
(d) Compilation phases

Phase	Errors Detected
Lexical Analysis	Invalid token (@)
Syntax Analysis	Missing ;, missing :
Semantic Analysis	Incorrect function call
Intermediate Code Generation	Cannot generate intermediate code if syntax/semantic errors remain
Code Optimization	Removes redundant operations such as +0
Runtime	Division by zero, invalid pointer access, array overflow

(d) Correct Program

```
#include <stdio.h>

int factorial(int n)
{
   if(n <= 1)
      return 1;
   return n * factorial(n - 1);
}

int main()
{
   int num = 5;
```

```
    int value = 10;

    int result = factorial(num);

    if(result >= 100)
        printf("Large Number\n");

    switch(result)
    {
        case 24:
            printf("Factorial of 4\n");
            break;

        case 120:
            printf("Factorial of 5\n");
            break;

        default:
            printf("Other Value\n");
    }

    int numbers[5]={2,4,6,8,10};
    printf("%d\n", numbers[4]);

    int *ptr=&value;
    printf("%d\n", *ptr);

    int a=50,b=5;
    int c=a/b;
    printf("%d\n", c);

    int sum=result+value;
    printf("%d\n", sum);

    int i=1;
    while(i<=5)
    {
        printf("%d\n", i);
        i++;
    }

    return 0;
}
```



(e) Two compiler improvements

1. Static Analysis to detect possible runtime issues (null pointers, division by zero, array bounds).
2. Advanced optimization and data-flow analysis to remove redundant code and warn about infinite loops.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand